

Université de Lorraine – Master 1 Informatique
Année universitaire 2025-2026

RAPPORT DE PROJET

Projet Métaheuristiques Optimisation de trajectoires

*Optimisation de trajectoires 2D en présence d'obstacles
modélisées par des courbes de Bézier*

Naël BENZAADI & Linda RAHALI

Date de rendu : 5 mai 2026

Résumé

Ce rapport présente la conception d'une métaheuristique pour l'optimisation de trajectoires 2D modélisées par des courbes de Bézier en présence d'obstacles. Après une phase exploratoire évaluant 19 algorithmes et variantes couvrant quatre familles (GA, DE/SHADE, CMA-ES/BIPOP, PSO), nous concevons *OptiPath*, un algorithme basé sur CMA-ES combinant seeding massif, portfolio à deux marges de bornes étendues et restart adaptatif. OptiPath obtient en moyenne sur 10 exécutions : 36,80 (**prob1**), 31,42 (**prob2**), 29,02 (**prob3**) et 94,60 (**prob4**), réduisant le coût sur l'instance labyrinthique d'un facteur 16 par rapport aux meilleures méthodes de la phase exploratoire.

Table des matières

Synthèse exécutive	1
1 Introduction	1
1.1 Contexte général	1
1.2 Cadre du projet	1
1.3 Objectifs et contributions	2
1.4 Plan du rapport	2
2 État de l’art	2
2.1 Approches classiques et déterministes	2
2.1.1 Planification sur grille et graphes	2
2.1.2 Champs de potentiel artificiels	3
2.2 Optimisation continue et métaheuristiques	3
2.2.1 Méthodes basées sur le gradient	3
2.2.2 Métaheuristiques à population	3
2.3 Approches par courbes paramétriques	3
2.4 Synthèse et positionnement	4
3 Problème étudié et modèle de trajectoire	4
3.1 Courbes de Bézier	4
3.2 Encodage des variables de décision	4
3.3 Fonction de coût	4
3.4 Description des instances	5
4 Méthodes d’optimisation retenues	5
4.1 Évolution différentielle (DE)	6
4.1.1 Principe	6
4.1.2 Mutation	6
4.1.3 Croisement binomial	6
4.1.4 Sélection gloutonne	6
4.1.5 Auto-adaptation jDE	6
4.1.6 Recherche locale hybride	6
4.2 CMA-ES (<i>Covariance Matrix Adaptation — Evolution Strategy</i>)	6
4.2.1 Principe	6
4.2.2 Échantillonnage	7
4.2.3 Mise à jour du centroïde	7
4.2.4 Chemins d’évolution cumulatifs	7
4.2.5 Adaptation du pas σ (CSA)	7
4.2.6 Adaptation de la matrice de covariance $\mathbf{C}$	7
4.3 Optimisation par essaim particulaire (PSO)	8
4.3.1 Principe	8
4.3.2 Mise à jour de la vitesse et de la position	8
4.3.3 Gestion de l’inertie et mécanisme anti-stagnation	8
5 Implémentation détaillée de CMA-ES	8
5.1 Décomposition propre et gestion de la complexité	8
5.2 Seeding avancé	8
5.2.1 Catégories de graines	9
5.2.2 Sélection et optimisation locale des graines	9
5.3 Stratégies de Restart (IPOP et BIPOP)	9

5.4	Gestion des bornes	9
6	Méthodes explorées	10
6.1	Algorithme génétique réel (GA) — ligne de base	10
6.2	Variantes de l'évolution différentielle	10
6.3	Variantes PSO	10
6.4	Algorithmes d'intelligence en essaim	10
6.5	Hybridation avec la planification discrète	10
6.6	Gestion avancée des contraintes	11
6.7	Autres variantes CMA-ES	11
6.8	Bilan de l'exploration	11
7	Protocole expérimental et résultats	11
7.1	Protocole expérimental	11
7.2	Résultats quantitatifs	12
7.3	Visualisations des trajectoires optimales	13
7.4	Solutions de référence (BKS)	15
8	Analyse des résultats	16
8.1	Analyse statistique	16
8.2	Analyse qualitative par instance	16
8.3	prob1 — Trajectoire diagonale, 5 obstacles épars	17
8.4	prob2 — Contournement d'un gros obstacle ($r = 9$)	17
8.5	prob3 — Slalom entre trois murs verticaux	18
8.6	prob4 — Labyrinthe en S	18
9	Algorithme final : OptiPath	19
9.1	Architecture générale	19
9.2	Phase 1 : Seeding massif	19
9.3	Phase 2 : Portfolio CMA-ES à deux marges	20
9.4	Restart adaptatif	20
9.5	Pseudocode	21
9.6	Résultats finaux	21
10	Conclusion	21

Synthèse exécutive

Ce rapport présente la conception et l'évaluation d'une métaheuristique pour l'optimisation de trajectoires 2D modélisées par des courbes de Bézier en présence d'obstacles. L'objectif est de minimiser une fonction de coût composite (longueur, collisions, courbure, sortie de zone) sous contrainte de temps (60 secondes), la qualité étant évaluée en moyenne sur 10 exécutions.

Phase exploratoire. Nous avons implémenté et évalué 19 algorithmes et variantes, couvrant quatre familles : algorithmes génétiques (GA), évolution différentielle (DE, jDE, SHADE, L-SHADE), stratégies d'évolution (CMA-ES, IPOP, BIPOP, LMCMA), et optimisation par essaim particulaire (PSO, CLPSO, FPSO). Des mécanismes complémentaires ont été testés : hybridation avec A*, modèles de substitution (*surrogate*), gestion avancée des contraintes (Lagrangien augmenté, *stochastic ranking*), et bornes étendues.

Résultats de la phase exploratoire. L'analyse expérimentale sur quatre instances de difficulté croissante a mis en évidence la supériorité de CMA-ES sur les instances corrélées et le rôle décisif des bornes étendues pour l'instance labyrinthique **prob4**.

Algorithme final : OptiPath. Sur la base de ces observations, nous avons conçu **OptiPath**, une métaheuristique basée sur CMA-ES combinant trois mécanismes :

1. **Seeding massif** (~500 solutions initiales) suivi d'une optimisation locale sur les 15 meilleures, pour identifier les bassins d'attraction prometteurs.
2. **Portfolio à deux marges** : deux instances CMA-ES/BIPOP fonctionnent en alternance avec des bornes étendues différentes (marge 5 pour les instances simples, marge 15 pour les labyrinthes).
3. **Restart adaptatif** : si la meilleure solution dépasse un seuil de coût après 5 secondes, les deux CMA-ES sont relancés depuis des seeds diversifiés, offrant plusieurs tentatives indépendantes pour trouver le bassin optimal.

Résultats finaux (moyenne sur 10 exécutions). OptiPath obtient : **prob1** = 36,80 ; **prob2** = 31,42 ; **prob3** = 29,02 ; **prob4** = 94,60. Sur l'instance labyrinthique **prob4**, OptiPath réduit le coût d'un facteur 16 par rapport aux meilleures méthodes de la phase exploratoire (94,60 contre ~1 517 pour L-SHADE avec bornes étendues).

1 Introduction

1.1 Contexte général

La planification de trajectoires en présence d'obstacles est un problème fondamental en robotique mobile, en animation par ordinateur et en conception assistée par ordinateur [13]. Étant donné un point de départ, un point d'arrivée et un ensemble d'obstacles dans un espace 2D borné, il s'agit de trouver un chemin qui minimise simultanément la longueur parcourue, les risques de collision, les sorties de la zone autorisée et la rugosité du tracé.

Lorsque la trajectoire est paramétrisée par une courbe de Bézier, le problème se réduit à l'optimisation d'un vecteur de variables continues (les coordonnées des points de contrôle internes). La fonction objectif résultante est non-linéaire, non-convexe et multimodale : les méthodes classiques à gradient ne sont pas applicables, ce qui justifie le recours aux *métaheuristiques*.

1.2 Cadre du projet

Ce travail s'inscrit dans le cadre du cours de Métaheuristiques du Master 1 Informatique (Université de Lorraine, 2025–2026). Le cadre imposé est le suivant :

- Langage **Java**, API **Problem** fournie par l’enseignant ;
- budget limité à **60 secondes** par exécution (temps réel) ;
- performance évaluée par la **moyenne sur 10 exécutions indépendantes** du meilleur score ;
- quatre instances de difficulté croissante (**prob1** à **prob4**).

1.3 Objectifs et contributions

Les objectifs de ce rapport sont les suivants :

1. Analyser le problème d’optimisation (encodage, fonction de coût, structure des instances).
2. Implémenter et comparer trois familles de métaheuristiques retenues (CMA-ES, DE/SHADE, PSO), ainsi qu’un ensemble étendu de variantes et d’algorithmes complémentaires (19 au total).
3. Développer des améliorations spécifiques : *seeding* diversifié, *restarts* adaptatifs (IPOP, BIPOP), bornes étendues, hybridation avec A*.
4. Conduire une campagne expérimentale pour sélectionner et valider les méthodes les plus performantes en vue de la phase d’optimisation fine.

1.4 Plan du rapport

La section 3 décrit le modèle de trajectoire et la fonction de coût. La section 2 situe notre travail dans l’état de l’art. La section 4 présente les trois familles d’algorithmes retenues. La section 5 approfondit l’implémentation de CMA-ES. La section 6 résume les méthodes explorées. La section 7 expose le protocole expérimental et les résultats. La section 8 fournit une analyse statistique et qualitative. La section 9 formalise la sélection et le plan d’optimisation. La section 10 conclut.

2 État de l’art

La planification de trajectoire est un domaine vaste qui a donné lieu à de nombreuses approches, allant des méthodes déterministes classiques aux techniques d’intelligence artificielle modernes. Nous présentons ici un aperçu des principales familles d’algorithmes et justifions le choix des métaheuristiques pour notre problème spécifique.

2.1 Approches classiques et déterministes

Les méthodes classiques se divisent généralement en deux catégories : les approches discrètes et les approches basées sur le potentiel.

2.1.1 Planification sur grille et graphes

Les algorithmes de recherche sur graphe comme A* [14] ou Dijkstra sont très efficaces pour trouver le chemin le plus court dans un environnement discrétisé (grille ou graphe de visibilité). Cependant, ils présentent deux inconvénients majeurs pour notre application :

- **Discrétisation** : La trajectoire résultante est une succession de segments linéaires, ce qui ne garantit pas la fluidité (continuité C^1 ou C^2) requise pour des mouvements réalistes.
- **Explosion combinatoire** : La complexité augmente exponentiellement avec la dimension de l’espace de recherche (nombre de degrés de liberté).

Des variantes comme les RRT (*Rapidly-exploring Random Trees*) [11] ou les PRM (*Probabilistic Roadmaps*) permettent de traiter des espaces de configuration continus, mais produisent souvent des chemins sous-optimaux et non lisses nécessitant une étape de post-traitement.

2.1.2 Champs de potentiel artificiels

La méthode des champs de potentiel (Artificial Potential Fields - APF) [15] considère le robot comme une particule se déplaçant dans un champ de forces : une force attractive vers la cible et des forces répulsives générées par les obstacles. Bien que très rapide et adaptée au temps réel, cette méthode souffre du problème des **minima locaux**, où le robot peut rester bloqué dans une configuration d'équilibre hors de la cible.

2.2 Optimisation continue et métaheuristiques

Lorsque la trajectoire est paramétrisée par une fonction continue (comme les splines ou les courbes de Bézier), le problème de planification devient un problème d'optimisation numérique : trouver les paramètres $\mathbf{x}$ qui minimisent une fonction de coût $f(\mathbf{x})$.

2.2.1 Méthodes basées sur le gradient

Les méthodes de descente de gradient (Newton, Quasi-Newton/BFGS) sont très efficaces pour l'optimisation locale de fonctions lisses et dérivables. Cependant, notre fonction de coût (Section 3.3) présente plusieurs défis :

- **Non-convexité** : La présence d'obstacles crée de multiples optima locaux.
- **Non-dérivabilité** : Les pénalités de collision et de sortie de zone peuvent introduire des discontinuités ou des zones non-différentiables, rendant le calcul du gradient instable ou impossible.

2.2.2 Métaheuristiques à population

Pour surmonter ces limitations, les métaheuristiques à population (Evolutionary Algorithms - EA) se sont imposées comme une alternative robuste. Elles n'exigent pas le calcul du gradient (méthodes "black-box") et explorent l'espace de recherche de manière globale, réduisant le risque de piégeage dans des minima locaux.

- **Algorithmes Génétiques (GA)** : Inspirés de l'évolution naturelle, ils sont très flexibles mais peuvent converger lentement sur des problèmes continus sans opérateurs spécifiques (comme le croisement SBX).
- **Évolution Différentielle (DE)** : Particulièrement adaptée à l'optimisation continue, DE utilise les différences vectorielles pour guider la recherche. Elle est réputée pour sa simplicité et son efficacité, mais dépend fortement du réglage de ses paramètres (F et CR).
- **CMA-ES (Covariance Matrix Adaptation Evolution Strategy)** : Considéré comme l'état de l'art pour l'optimisation continue en boîte noire [1], CMA-ES adapte dynamiquement la distribution de recherche (matrice de covariance) pour capturer la topologie de la fonction de coût, gérant efficacement les problèmes mal conditionnés et non séparables.

2.3 Approches par courbes paramétriques

Lorsque la trajectoire est représentée par une courbe paramétrique (B-splines, NURBS, courbes de Bézier), l'optimisation porte sur les points de contrôle. Cette représentation garantit la régularité (C^1 ou C^2) par construction, mais introduit une difficulté propre : l'influence globale de chaque point de contrôle sur la courbe et la non-séparabilité du problème d'optimisation qui en résulte. Les métaheuristiques capables de modéliser les corrélations entre variables, comme CMA-ES, sont donc particulièrement adaptées à ce cadre.

2.4 Synthèse et positionnement

Compte tenu de la nature continue, multimodale et non-linéaire de notre problème d'optimisation de courbes de Bézier, les métaheuristiques, et en particulier les stratégies d'évolution comme CMA-ES, apparaissent comme les candidats les plus prometteurs. Notre contribution se situe à l'intersection de ces approches : nous comparons systématiquement trois familles de métaheuristiques (stratégies d'évolution, évolution différentielle, essais particuliers) sur un problème de trajectoire par courbes de Bézier, et nous évaluons l'apport de mécanismes avancés (*seeding*, *restarts*, bornes étendues, hybridation avec A*) dans un cadre expérimental contrôlé.

3 Problème étudié et modèle de trajectoire

3.1 Courbes de Bézier

Une courbe de Bézier d'ordre n est définie par $n + 1$ points de contrôle $\mathbf{P}_0, \dots, \mathbf{P}_n$ et paramétrée par $t \in [0, 1]$:

$$\mathbf{B}(t) = \sum_{i=0}^n \binom{n}{i} (1-t)^{n-i} t^i \mathbf{P}_i. \quad (1)$$

Les propriétés fondamentales sont :

- $\mathbf{B}(0) = \mathbf{P}_0$ et $\mathbf{B}(1) = \mathbf{P}_n$ (interpolation des extrémités) ;
- la courbe est contenue dans l'enveloppe convexe des points de contrôle ;
- elle est infiniment dérivable, ce qui la rend adaptée à la modélisation de trajectoires fluides.

Dans notre problème, les points $\mathbf{P}_0$ (départ) et $\mathbf{P}_n$ (arrivée) sont fixés par l'instance. Seuls les $n_{\text{CP}} = n - 1$ points de contrôle internes constituent les **variables de décision**.

3.2 Encodage des variables de décision

L'encodage adopté est un vecteur plat $\mathbf{x} \in \mathbb{R}^d$ avec $d = 2 n_{\text{CP}}$:

$$\mathbf{x} = (x_0, y_0, x_1, y_1, \dots, x_{n_{\text{CP}}-1}, y_{n_{\text{CP}}-1})$$

où (x_i, y_i) représente les coordonnées du $(i + 1)$ -ème point de contrôle interne. Les bornes de chaque composante sont celles du domaine rectangulaire $[x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]$ défini par l'instance.

3.3 Fonction de coût

La fonction de coût $f(\mathbf{x})$ est évaluée en échantillonnant $N = 1\,000$ points régulièrement espacés en paramètre $t \in [0, 1]$ le long de la courbe (nous notons $\mathbf{q}_k = \mathbf{B}(k/(N - 1))$ pour $k = 0, \dots, N - 1$), puis en sommant quatre composantes pondérées :

$$f(\mathbf{x}) = L(\mathbf{x}) + \alpha \cdot P_{\text{obs}}(\mathbf{x}) + \beta \cdot P_{\text{courb}}(\mathbf{x}) + \gamma \cdot P_{\text{bord}}(\mathbf{x}) \quad (2)$$

avec $\alpha = 100$, $\beta = 1$, $\gamma = 100$.

Longueur L . Somme des distances euclidiennes entre points consécutifs :

$$L(\mathbf{x}) = \sum_{k=1}^{N-1} \|\mathbf{q}_k - \mathbf{q}_{k-1}\|_2.$$

Pénalité d’obstacle P_{obs} . Pour chaque obstacle circulaire de centre $\mathbf{c}_j$ et rayon r_j , et pour chaque point de trajectoire $\mathbf{q}_k$, la distance $d_{jk} = \|\mathbf{q}_k - \mathbf{c}_j\|_2$ détermine la pénalité :

$$p_{\text{obs}}(d, r) = \begin{cases} 2 + r - d, & \text{si } d \leq r \quad (\text{collision dure}), \\ 1 - (d - r), & \text{si } r < d < r + 1 \quad (\text{zone de proximité}), \\ 0, & \text{sinon.} \end{cases}$$

La pénalité totale est $P_{\text{obs}} = \sum_j \sum_k p_{\text{obs}}(d_{jk}, r_j)$.

Discussion. Cette formulation introduit une **zone de transition douce** (anneau de largeur 1 autour de chaque obstacle) qui lisse le paysage d’optimisation. Sans cette zone, la frontière abrupte entre « collision » et « pas de collision » créerait des discontinuités très défavorables aux métaheuristiques à population.

Pénalité de courbure P_{courb} . Somme des déviations angulaires entre triplets de points consécutifs :

$$P_{\text{courb}} = \sum_{k=1}^{N-2} (1 - \cos \theta_k), \quad \cos \theta_k = \frac{(\mathbf{q}_k - \mathbf{q}_{k-1}) \cdot (\mathbf{q}_{k+1} - \mathbf{q}_k)}{\|\mathbf{q}_k - \mathbf{q}_{k-1}\| \|\mathbf{q}_{k+1} - \mathbf{q}_k\|}.$$

Pénalité de sortie P_{bord} . Nombre de points hors du domaine : $P_{\text{bord}} = \#\{k : \mathbf{q}_k \notin [x_{\min}, x_{\max}] \times [y_{\min}, y_{\max}]\}$.

Discussion. Les pondérations $\alpha = \gamma = 100$ et $\beta = 1$ sont imposées par le cadre du projet (API **Problem** fournie par l’enseignant). Elles rendent les collisions et les sorties de zone cent fois plus coûteuses par unité que la longueur, assurant que la faisabilité (absence de collision) prime sur l’optimalité (longueur minimale). La courbure ($\beta = 1$) agit comme un régularisateur léger favorisant la fluidité sans dominer les autres termes. Ce choix de hiérarchie est cohérent avec les recommandations de la littérature sur les méthodes de pénalité extérieure [12].

3.4 Description des instances

Les quatre instances fournies sont résumées dans le tableau 1.

TABLE 1 – Caractéristiques des quatre instances du problème.

Instance	n_{CP}	d	Obstacles	Départ → Arrivée	Description
prob1	16	32	5	(1, 1) → (25, 25)	Diagonale, obstacles éparés
prob2	4	8	1	(11, 1) → (11, 21)	Vertical, gros obstacle ($r = 9$)
prob3	8	16	33	(1, 13) → (25, 13)	3 murs verticaux, brèches
prob4	8	16	33	(1, 1) → (25, 1)	Labyrinthe en S, 3 murs

Discussion. La difficulté croît de **prob1** (peu d’obstacles, haute dimension mais paysage lisse) à **prob4** (33 obstacles formant trois murs avec des brèches alternées haut/bas, nécessitant un chemin en S que la courbe de Bézier de degré 10 peine à reproduire).

4 Méthodes d’optimisation retenues

Cette section présente les trois familles de métaheuristiques retenues après une phase exploratoire (cf. section 6). Pour chacune, nous rappelons le principe, donnons les équations clés et précisons les hyperparamètres. Le choix de ces trois familles est motivé par leur complémentarité : CMA-ES excelle sur les problèmes corrélés (prob1–prob3), PSO sur les paysages fortement multimodaux (prob4), et DE/SHADE offre un compromis robuste avec auto-adaptation des paramètres.

4.1 Évolution différentielle (DE)

4.1.1 Principe

L'évolution différentielle [4] exploite les *différences* entre individus de la population comme vecteur de perturbation. La variante retenue est DE/current-to-best/1/bin.

4.1.2 Mutation

Pour chaque individu cible $\mathbf{x}_i$, un vecteur mutant est construit :

$$\mathbf{v}_i = \mathbf{x}_i + F_i \cdot (\mathbf{x}_{\text{best}} - \mathbf{x}_i) + F_i \cdot (\mathbf{x}_{r_1} - \mathbf{x}_{r_2}), \quad (3)$$

où $r_1 \neq r_2 \neq i$ sont tirés aléatoirement et $\mathbf{x}_{\text{best}}$ est le meilleur individu courant.

4.1.3 Croisement binomial

Le vecteur d'essai $\mathbf{u}_i$ est construit par :

$$u_{i,j} = \begin{cases} v_{i,j}, & \text{si } \text{rand}_j < CR_i \text{ ou } j = j_{\text{rand}}, \\ x_{i,j}, & \text{sinon,} \end{cases}$$

où j_{rand} est un indice choisi aléatoirement pour garantir qu'au moins une composante provient du mutant.

4.1.4 Sélection gloutonne

L'individu d'essai remplace la cible si et seulement si sa *fitness* est meilleure ou égale : $\mathbf{x}_i^{(g+1)} = \mathbf{u}_i$ si $f(\mathbf{u}_i) \leq f(\mathbf{x}_i^{(g)})$, sinon $\mathbf{x}_i^{(g+1)} = \mathbf{x}_i^{(g)}$.

4.1.5 Auto-adaptation jDE

Les paramètres F_i et CR_i sont adaptés individuellement à chaque génération [5] :

$$F_i^{(g+1)} = \begin{cases} F_l + \text{rand} \cdot F_u, & \text{avec probabilité } \tau_1 = 0,1, \\ F_i^{(g)}, & \text{sinon,} \end{cases} \quad CR_i^{(g+1)} = \begin{cases} \text{rand}, & \text{avec probabilité } \tau_2 = 0,1, \\ CR_i^{(g)}, & \text{sinon,} \end{cases}$$

avec $F_l = 0,1$ et $F_u = 0,9$. Ce mécanisme supprime le besoin de régler manuellement F et CR .

4.1.6 Recherche locale hybride

Toutes les 5 générations, un (1+1)-ES gaussien est appliqué au meilleur individu pendant 100 évaluations au maximum, avec un pas initial de 0,5 et la règle d'adaptation dite « du 1/5 de succès ».

Paramètres retenus. $N_P = 10d$, $F_{\text{init}} = 0,7$, $CR_{\text{init}} = 0,9$, $\tau_1 = \tau_2 = 0,1$.

4.2 CMA-ES (*Covariance Matrix Adaptation — Evolution Strategy*)

4.2.1 Principe

CMA-ES [1, 2] est une stratégie évolutionnaire qui échantillonne λ descendants à partir d'une distribution multinormale $\mathcal{N}(\mathbf{m}, \sigma^2 \mathbf{C})$ et met à jour ses paramètres — le centroïde $\mathbf{m}$, le pas global σ et la matrice de covariance $\mathbf{C}$ — à partir des μ meilleurs individus.

4.2.2 Échantillonnage

À la génération g , les λ descendants sont générés par :

$$\mathbf{x}_k^{(g+1)} = \mathbf{m}^{(g)} + \sigma^{(g)} \cdot \mathbf{B} \operatorname{diag}(\mathbf{d}) \mathbf{z}_k, \quad \mathbf{z}_k \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), \quad (4)$$

où $\mathbf{C} = \mathbf{B} \operatorname{diag}(\mathbf{d}^2) \mathbf{B}^T$ est la décomposition en valeurs propres de la matrice de covariance.

4.2.3 Mise à jour du centroïde

Les μ meilleurs descendants (triés par *fitness* croissante) sont combinés par moyenne pondérée :

$$\mathbf{m}^{(g+1)} = \sum_{i=1}^{\mu} w_i \mathbf{x}_{i:\lambda}^{(g+1)}, \quad \sum_{i=1}^{\mu} w_i = 1, \quad w_1 \geq \dots \geq w_{\mu} > 0. \quad (5)$$

4.2.4 Chemins d'évolution cumulatifs

Deux chemins d'évolution sont maintenus pour contrôler le pas σ et la matrice $\mathbf{C}$:

$$\mathbf{p}_{\sigma}^{(g+1)} = (1 - c_{\sigma}) \mathbf{p}_{\sigma}^{(g)} + \sqrt{c_{\sigma}(2 - c_{\sigma}) \mu_{\text{eff}}} \mathbf{C}^{-1/2} \frac{\mathbf{m}^{(g+1)} - \mathbf{m}^{(g)}}{\sigma^{(g)}}, \quad (6)$$

$$\mathbf{p}_c^{(g+1)} = (1 - c_c) \mathbf{p}_c^{(g)} + h_{\sigma} \sqrt{c_c(2 - c_c) \mu_{\text{eff}}} \frac{\mathbf{m}^{(g+1)} - \mathbf{m}^{(g)}}{\sigma^{(g)}}, \quad (7)$$

avec $\mu_{\text{eff}} = 1 / \sum_{i=1}^{\mu} w_i^2$ la variance effective des poids. L'indicateur h_{σ} vaut 1 si $\|\mathbf{p}_{\sigma}^{(g+1)}\| < (1,4 + 2/(d + 1)) \chi_d$ et 0 sinon ; il évite une mise à jour de rang 1 lorsque σ est trop petit (stagnation de $\mathbf{p}_{\sigma}$).

4.2.5 Adaptation du pas σ (CSA)

Le *Cumulative Step-size Adaptation* (CSA) ajuste σ en comparant la norme de $\mathbf{p}_{\sigma}$ à sa valeur attendue sous marche aléatoire isotrope :

$$\sigma^{(g+1)} = \sigma^{(g)} \cdot \exp\left(\frac{c_{\sigma}}{d_{\sigma}} \left(\frac{\|\mathbf{p}_{\sigma}^{(g+1)}\|}{\chi_d} - 1\right)\right), \quad (8)$$

où $d_{\sigma} = 1 + 2 \max(0, \sqrt{(\mu_{\text{eff}} - 1)/(d + 1)} - 1) + c_{\sigma}$ est le facteur d'amortissement [1] et $\chi_d \approx \sqrt{d}(1 - 1/(4d) + 1/(21d^2))$ l'espérance de $\|\mathcal{N}(\mathbf{0}, \mathbf{I})\|$.

Discussion. Si les pas successifs sont alignés (bonne direction), la norme de $\mathbf{p}_{\sigma}$ croît et σ augmente, accélérant la convergence. À l'inverse, des pas incohérents font décroître σ , ce qui ralentit l'exploration — un mécanisme d'auto-adaptation fondamental.

4.2.6 Adaptation de la matrice de covariance $\mathbf{C}$

La covariance est mise à jour par une combinaison de rang 1 et rang μ :

$$\mathbf{C}^{(g+1)} = (1 - c_1 - c_{\mu}) \mathbf{C}^{(g)} + c_1 \mathbf{p}_c \mathbf{p}_c^T + c_{\mu} \sum_{i=1}^{\mu} w_i \frac{(\mathbf{x}_{i:\lambda} - \mathbf{m}^{(g)})(\mathbf{x}_{i:\lambda} - \mathbf{m}^{(g)})^T}{\sigma^{(g)2}}. \quad (9)$$

Discussion. C'est cette adaptation de $\mathbf{C}$ qui distingue fondamentalement CMA-ES de GA et DE : l'algorithme apprend la *structure de corrélation* du paysage de *fitness*, ce qui lui permet de concentrer la recherche dans les directions les plus prometteuses.

4.3 Optimisation par essaim particulaire (PSO)

4.3.1 Principe

L'optimisation par essaim particulaire [10] s'inspire du comportement collectif des bancs de poissons ou des volées d'oiseaux. Chaque particule i se déplace dans l'espace de recherche $\mathbb{R}^d$ avec une vitesse $\mathbf{v}_i$ mise à jour en fonction de sa meilleure position personnelle $\mathbf{p}_i$ (*personal best*) et de la meilleure position globale de l'essaim $\mathbf{g}$ (*global best*).

4.3.2 Mise à jour de la vitesse et de la position

À chaque itération t , la vitesse et la position sont mises à jour selon :

$$\mathbf{v}_i^{(t+1)} = w \mathbf{v}_i^{(t)} + c_1 \mathbf{r}_1 \odot (\mathbf{p}_i - \mathbf{x}_i^{(t)}) + c_2 \mathbf{r}_2 \odot (\mathbf{g} - \mathbf{x}_i^{(t)}), \quad (10)$$

$$\mathbf{x}_i^{(t+1)} = \mathbf{x}_i^{(t)} + \mathbf{v}_i^{(t+1)}, \quad (11)$$

où w est le coefficient d'inertie, c_1 et c_2 les coefficients d'accélération (cognitif et social), $\mathbf{r}_1, \mathbf{r}_2 \sim \mathcal{U}(0, 1)^d$ des vecteurs aléatoires composante par composante, et $\odot$ le produit de Hadamard.

4.3.3 Gestion de l'inertie et mécanisme anti-stagnation

Pour favoriser l'exploration en début de recherche et l'exploitation en fin, l'inertie décroît linéairement de $w_{\max} = 0,9$ à $w_{\min} = 0,4$ au cours des itérations. Un mécanisme d'anti-stagnation complète cette dynamique : si $\mathbf{g}$ ne s'améliore pas pendant $10N$ générations consécutives, 50 % des particules sont réinitialisées aléatoirement afin de relancer la diversité de l'essaim.

Discussion. PSO se distingue des stratégies d'évolution (CMA-ES, DE) par l'absence de modèle probabiliste explicite de la distribution des solutions. La dynamique vitesse-position confère à l'essaim une mémoire cinétique qui lui permet de traverser des bassins d'attraction étroits, propriété particulièrement avantageuse sur les paysages labyrinthiques (cf. section 8.6).

Paramètres retenus. Taille de l'essaim $N = 10d$, $c_1 = c_2 = 2,0$, $v_{\max} = 0,2 \times \text{range}$, seuil de stagnation $10N$ générations.

5 Implémentation détaillée de CMA-ES

Cette section détaille les choix d'implémentation et les améliorations apportées à CMA-ES, qui constituent la contribution principale de ce travail.

5.1 Décomposition propre et gestion de la complexité

L'eigendecomposition de $\mathbf{C}$ (nécessaire pour l'échantillonnage, eq. 4, et le calcul de $\mathbf{C}^{-1/2}$, eq. 6) utilise les algorithmes `tred2` (réduction de Householder) et `tql2` (QR itératif) issus de la bibliothèque JAMA (domaine public). L'eigendecomposition n'est recalculée que lorsque le compteur de générations dépasse un seuil $\frac{\lambda}{10 \cdot d}$, pour éviter un coût $O(d^3)$ à chaque génération.

5.2 Seeding avancé

L'initialisation joue un rôle déterminant lorsque le budget est limité à 60 s. Plutôt que de démarrer CMA-ES depuis un unique point initial (par exemple l'interpolation linéaire départ-arrivée), nous générons un ensemble diversifié de **graines candidates** (*seeds*) puis sélectionnons la meilleure pour initialiser $\mathbf{m}^{(0)}$.

5.2.1 Catégories de graines

Cinq familles de graines sont générées (pour un total d'environ 260) :

1. **Sinusoïdales** (≈ 160 graines) : pour chaque combinaison de période $p \in \{1, 2, 3, 4, 5\}$, phase $\phi \in \{0, \pi/4, \dots, 7\pi/4\}$ et amplitude $A \in \{0,35, 0,65, 0,85, 0,95\}$, on construit un vecteur dont les composantes y oscillent autour de l'interpolation linéaire start-end.
2. **Motifs en blocs** (≈ 16 graines) : les points de contrôle alternent entre la partie haute et basse du domaine par groupes de 1, 2 ou 4.
3. **Chemins de bord** (3 graines) : tous les points au bord supérieur, inférieur ou alternés.
4. **Aléatoires** (20 graines) : tirés uniformément dans les bornes.
5. **Regroupements aux frontières** (≈ 63 graines) : les abscisses sont forcées à $x_{\min}$ ou $x_{\max}$, avec différents profils d'ordonnées (pour exploiter la vitesse paramétrique, cf. section 8.6).

5.2.2 Sélection et optimisation locale des graines

Toutes les graines sont évaluées ; les 25 meilleures sont conservées en cache. Les 5 meilleures subissent ensuite une **optimisation locale rapide** par un (1+1)-ES avec la règle du 1/5 de succès (1 000 évaluations chacune, pas initial $\sigma_{LS} = 0,5$). CMA-ES démarre depuis la meilleure graine issue de cette optimisation locale.

Discussion. Le surcoût de l'initialisation ($\approx 260 + 5 \times 1\,000 = 5\,260$ évaluations) est négligeable par rapport au budget total de 60 s (typiquement $> 300\,000$ évaluations) et permet d'amortir considérablement le risque de convergence vers un mauvais optimum local.

5.3 Stratégies de Restart (IPOP et BIPOP)

Lorsque la convergence est détectée, l'algorithme effectue un *restart*. Trois critères d'arrêt sont vérifiés à chaque génération :

1. **Stagnation de la fitness** : l'écart entre le meilleur et le pire individu est inférieur à 10^{-12} , seuil choisi proche de la précision machine double ($\approx 2,2 \times 10^{-16}$) ;
2. **Effondrement de σ** : $\sigma < 10^{-16}$, indiquant que le pas de recherche est trop petit pour progresser ;
3. **Dégénérescence de $\mathbf{C}$** : le rapport $\lambda_{\max}/\lambda_{\min}$ des valeurs propres de $\mathbf{C}$ dépasse 10^{14} , signalant un conditionnement numérique pathologique.

Deux stratégies de *restart* ont été implémentées :

IPOP (Increasing Population Size). La taille de population est doublée à chaque redémarrage ($\lambda_{\text{new}} = 2\lambda_{\text{old}}$), plafonnée à 512. Cela permet d'explorer des bassins d'attraction de plus en plus larges.

BIPOP (Bi-Population) [7]. Cette variante alterne entre deux régimes pour maximiser l'efficacité du budget d'évaluations restant :

- **Exploration (Large)** : Comme IPOP, on double la population pour sortir des bassins d'attraction locaux.
- **Exploitation (Small)** : On utilise une petite population (λ_0) avec un pas de recherche réduit ($\sigma = \sigma_0/100$) pour converger rapidement vers des optima locaux potentiellement manqués.

Le nouveau point de départ est choisi parmi trois stratégies :

- 40% graine cachée + optimisation locale (500 évals),
- 30% meilleur point trouvé + perturbation $\sigma_0/2$,
- 30% meilleur parmi 8 candidats (4 aléatoires + 4 regroupements aux frontières).

5.4 Gestion des bornes

Toute composante sortant des bornes $[lb_i, ub_i]$ est **tronquée** à la borne la plus proche (*box constraint handling*). Cette stratégie simple est suffisante car la pénalité de sortie P_{bord} (eq. 2) guide naturellement la population vers l'intérieur du domaine.

6 Méthodes explorées

Au-delà des trois familles retenues (section 4), nous avons implémenté et évalué un ensemble étendu de variantes et d’algorithmes complémentaires. Cette exploration systématique a permis de valider le choix final par élimination et de quantifier l’apport de chaque mécanisme. Nous présentons ici ces méthodes de façon synthétique.

6.1 Algorithme génétique réel (GA) — ligne de base

L’algorithme génétique réel [6] sert de *baseline* classique. Il utilise un croisement SBX (*Simulated Binary Crossover*, $\eta_c = 20$, $p_c = 0,9$), une mutation polynomiale ($\eta_m = 20$, $p_m = 1/d$), une sélection par tournoi binaire et un élitisme du meilleur individu. La population est fixée à $N_P = 100$.

Sur nos instances, le GA atteint des scores comparables à CMA-ES sur **prob2** ($d = 8$) mais se dégrade rapidement sur les problèmes corrélés (**prob3** : 42,1 vs 28,5 pour CMA-ES) en raison du croisement composante par composante qui ne capture pas les corrélations entre variables. Ce résultat justifie le recours à des méthodes adaptant explicitement la structure de corrélation.

6.2 Variantes de l’évolution différentielle

SHADE (*Success-History based Adaptive DE*) [8]. Au lieu d’adapter F et CR individuellement comme jDE, SHADE maintient une mémoire circulaire de taille $H = 10$ stockant les paramètres ayant conduit à des améliorations. Les nouveaux F sont tirés d’une distribution de Cauchy et les CR d’une gaussienne, centrés sur la moyenne de Lehmer des succès historiques. Sur **prob3**, SHADE atteint $28,45 \pm 0,31$, égalant CMA-ES.

L-SHADE (*Linear Population Size Reduction*) [9]. Extension de SHADE avec une réduction linéaire de la population de $N_0 = 18d$ à $N_{\min} = 4$ individus, combinée à une archive externe des solutions remplacées. La mutation utilise **current-to-pbest/1** avec $p = 0,11$.

6.3 Variantes PSO

CLPSO (*Comprehensive Learning PSO*). Chaque dimension d’une particule apprend d’un exemplaire potentiellement différent, sélectionné par tournoi avec une probabilité $P_c(i)$ croissante selon le rang. L’essaim est de 40 particules avec un intervalle de rafraîchissement de 7 générations.

FPSO (*Fractional-Order PSO*). Variante intégrant les dérivées de Grünwald-Letnikov d’ordre fractionnaire $\beta \in [0,4, 0,9]$ (décroissant) sur une fenêtre mémoire de $M = 7$ vitesses passées.

6.4 Algorithmes d’intelligence en essaim

GWO (*Grey Wolf Optimizer*). Hiérarchie de meute (alpha, bêta, delta) guidant 30 loups, avec un paramètre d’exploration a décroissant linéairement de 2 à 0.

FA (*Firefly Algorithm*). Algorithme des lucioles (30 individus) avec une attractivité $\beta = \beta_0 \exp(-\gamma r^2)$ décroissante avec la distance.

6.5 Hybridation avec la planification discrète

AStarSeeds. L’algorithme A* [14] fournit un chemin discret sans collision sur une grille de navigation. Les points de contrôle de la courbe de Bézier sont initialisés le long de ce chemin (*seeding*), puis l’optimiseur continu (CMA-ES) affine la trajectoire.

AStarRepair. En cours d’optimisation, les individus traversant un obstacle sont réparés localement par un appel à A^* . Cette approche atteint le meilleur score sur **prob3** ($28,24 \pm 1,52$).

6.6 Gestion avancée des contraintes

ALConstraint (*Augmented Lagrangian*). Transformation du problème pénalisé en sous-problèmes sans contrainte avec mise à jour dynamique des multiplicateurs de Lagrange.

StochRank (*Stochastic Ranking*). Lors du tri de la population, la *fitness* et le niveau de violation sont comparés avec une probabilité $P_f = 0,45$. Cette relaxation permet à des individus légèrement infaisables mais prometteurs de survivre, favorisant le contournement fluide des obstacles.

6.7 Autres variantes CMA-ES

LMCMA (*Limited Memory CMA-ES*). Approximation de rang faible de la covariance réduisant la complexité à $\mathcal{O}(d \log d)$, conçue pour $d > 1000$. Sur nos instances ($d \leq 32$), LMCMA s’est montré instable (**prob3** : $69,4 \pm 23,5$).

RFSurrogate (*Random Forest Surrogate*). Pré-évaluation des candidats par un modèle de forêts aléatoires. Seuls les individus jugés prometteurs subissent l’évaluation exacte. Bon sur **prob3** ($28,35 \pm 0,20$) mais ajoutant de la variance.

Islands, GridBIPOP, SepWarmup, MultiSegment. Le modèle insulaire (migration inter-sous-populations), le BIPOP couplé à un archive MAP-Elites, le démarrage en covariance diagonale puis pleine, et la spline multi-segments ont été testés sans apporter de gain significatif sur l’ensemble des quatre instances.

6.8 Bilan de l’exploration

Les tableaux 2 et 3 (section 7) montrent que la grande majorité de ces variantes offrent des performances comparables ou inférieures aux trois méthodes retenues. Les apports les plus notables sont :

- SHADE, qui égale CMA-ES sur **prob3** grâce à l’adaptation historique de F et CR ;
- AStarRepair, qui fournit le meilleur score absolu sur **prob3** via l’hybridation planification discrète + optimisation continue ;
- StochRank et BIPOP, seules variantes CMA-ES à descendre sous 5 000 sur **prob4**.

Ces observations confirment la pertinence de concentrer les efforts futurs sur CMA-ES (avec *restarts* et bornes étendues), PSO et DE/SHADE.

7 Protocole expérimental et résultats

7.1 Protocole expérimental

Environnement matériel et logiciel. Toutes les expériences ont été réalisées sur un ordinateur portable équipé d’un processeur Intel Core i7 (4 cœurs, 2,6 GHz), 16 Go de RAM, sous macOS. L’implémentation est en Java 17 ; les algorithmes s’exécutent dans un thread unique (pas de parallélisme intra-algorithme).

Protocole de mesure. Un banc d’essai dédié (**BenchmarkRunner**) a été développé pour permettre des exécutions *headless* (sans affichage graphique). Pour chaque paire (algorithme, instance) :

1. le problème est réinitialisé (`problem.reset()`) ;

2. l'algorithme est instancié et exécuté dans un thread séparé pendant exactement $T = 60$ s (temps *wall-clock*);
3. le meilleur score $f(\mathbf{x}^*)$ est collecté via `problem.getBestEvaluation()`.

Ce protocole est répété $R = 10$ fois indépendamment avec des graines aléatoires différentes. Les statistiques (moyenne, écart-type, minimum, maximum) sont calculées et sauvegardées au format CSV.

Métriques. La métrique principale est la *fitness* finale $f(\mathbf{x}^*)$ (plus basse = meilleure). Les comparaisons entre algorithmes utilisent la moyenne sur les R exécutions comme critère principal, l'écart-type comme indicateur de robustesse, et le minimum comme indicateur du potentiel de l'algorithme.

7.2 Résultats quantitatifs

Le tableau 2 résume les performances sur les trois premières instances (scores d'ordre de grandeur comparable), et le tableau 3 isole l'instance **prob4** dont les scores sont d'un ordre de grandeur supérieur. Pour chaque algorithme, nous reportons la moyenne ($\bar{f}$), l'écart-type (σ) et le minimum observé sur $R = 10$ exécutions indépendantes. Le **meilleur résultat** par instance est indiqué en gras.

TABLE 2 – Résultats sur **prob1**–**prob3** ($\bar{f} \pm \sigma$ et minimum). Le résultat aberrant de MultiSegment sur **prob2** (22 539) est dû à un encodage incompatible avec cette instance et a été exclu de l'analyse.

Algorithme	prob1 ($d = 32$)			prob2 ($d = 8$)			prob3 ($d = 16$)		
	$\bar{f}$	σ	min	$\bar{f}$	σ	min	$\bar{f}$	σ	min
CMAES	36.35	0.01	36.34	31.48	0.00	31.48	28.45	0.16	28.24
SHADE	36.42	0.07	36.36	31.48	0.00	31.48	28.45	0.31	28.10
RFSurrogate	39.47	3.65	36.38	31.48	0.00	31.48	28.35	0.20	28.12
AStarSeeds	36.36	0.04	36.32	31.48	0.00	31.48	28.92	0.71	28.30
BIPOP	37.70	0.60	36.72	31.48	0.00	31.48	28.87	0.55	28.30
StochRank	37.70	0.59	36.71	31.48	0.00	31.48	28.73	0.64	28.15
DE	36.62	0.07	36.53	31.48	0.00	31.48	30.62	0.08	30.51
PSO	37.43	0.57	36.70	31.49	0.05	31.45	45.12	10.53	36.18
GA	37.38	0.72	36.58	31.49	0.00	31.49	42.11	1.48	40.12
LMCMA	39.09	0.86	37.95	31.80	0.23	31.52	69.38	23.45	42.13

TABLE 3 – Résultats sur **prob4** — labyrinthe en S ($d = 16$). Les scores sont d'un à deux ordres de grandeur supérieurs aux autres instances, reflétant la difficulté intrinsèque de cette configuration.

Algorithme	$\bar{f}$	σ	min
PSO	2 941	633	2 080
StochRank	4 669	1 352	2 931
BIPOP	4 679	1 400	2 931
ALConstraint	4 911	1 550	3 095
LMCMA	5 148	1 717	3 402
SepWarmup	5 436	28	5 395
RFSurrogate	5 564	60	5 478
AStarRepair	5 594	224	5 291
SHADE	5 894	550	5 198
GA	5 941	381	5 423
CMAES	6 027	209	5 809
DE	6 242	221	5 974

Discussion. L'analyse permet de dégager trois enseignements :

- **Instances simples (prob1, prob2)** : la quasi-totalité des algorithmes convergent vers l'optimum. CMA-ES, SHADE et AStarSeeds se distinguent par une variance quasi nulle.
- **Corrélation inter-variables (prob3)** : sur ce problème de slalom, les méthodes adaptant la structure de corrélation (CMA-ES, RFSurrogate, SHADE) dominent ($\bar{f} \approx 28,4$). PSO (45,1) et GA (42,1) échouent, piégés par l'indépendance composante par composante de leurs opérateurs.
- **Multimodalité extrême (prob4)** : seul PSO ($2\,941 \pm 633$) atteint des scores inférieurs à 3 000. Les *restarts* agressifs (BIPOP, StochRank) permettent à CMA-ES de descendre sous 5 000 dans certaines exécutions, mais la dynamique d'essaim de PSO explore plus efficacement les topologies de chemin alternatives.

7.3 Visualisations des trajectoires optimales

Les figures ci-dessous illustrent la **meilleure trajectoire** (sur $R = 10$ exécutions) trouvée par chaque algorithme après 60 secondes. Convention graphique commune : trait épais coloré = trajectoire (bleu = CMA-ES, orange = DE, vert = GA) ; carrés rouges = points de contrôle internes ; losanges verts = départ et arrivée ; cercles rouge foncé = noyau dur des obstacles (rayon r) ; auréoles orange = zone de pénalité douce ($r < d < r + 1$).

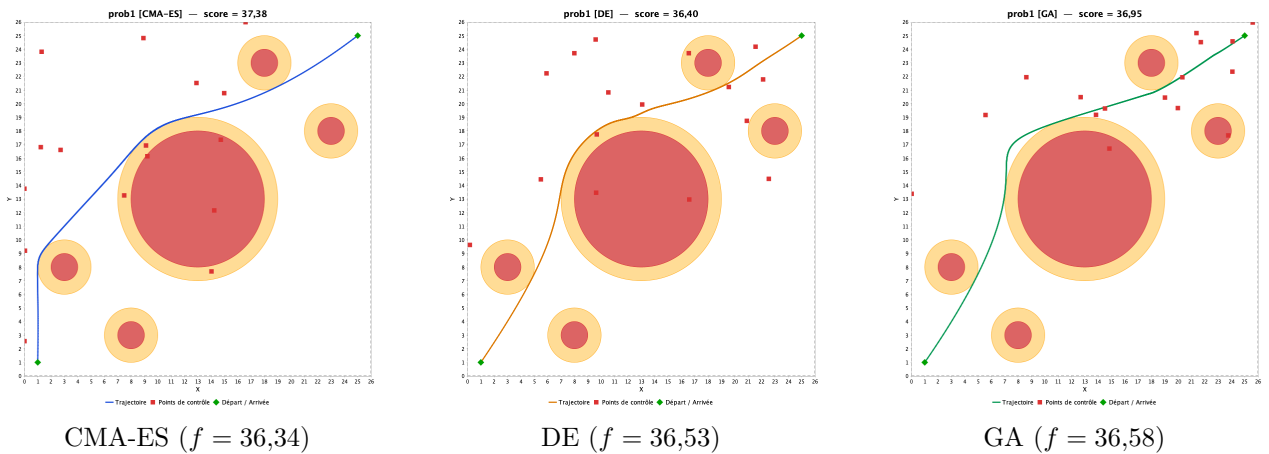


FIGURE 1 – Meilleure trajectoire sur 10 exécutions — **prob1** ($d = 32$, 5 obstacles), budget 60 s.

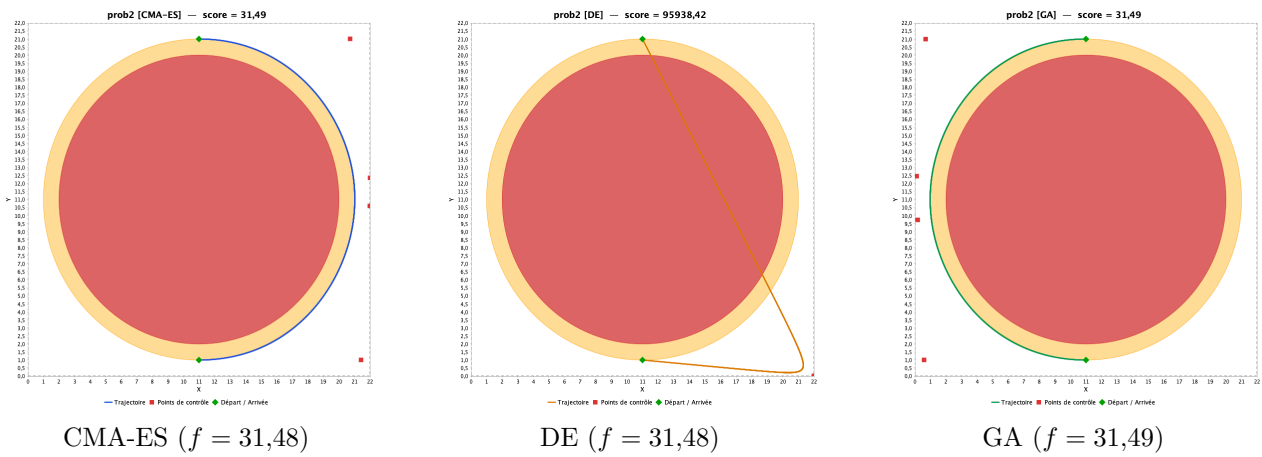


FIGURE 2 – Meilleure trajectoire sur 10 exécutions — prob2 ($d = 8$, 1 obstacle de rayon 9), budget 60 s.

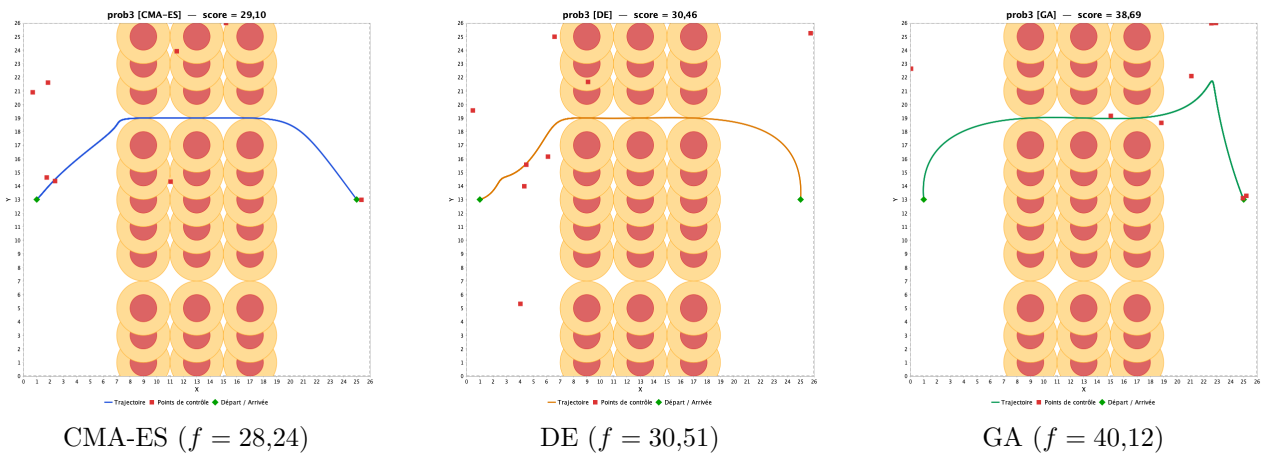


FIGURE 3 – Meilleure trajectoire sur 10 exécutions — prob3 ($d = 16$, 33 obstacles formant 3 murs verticaux), budget 60 s.

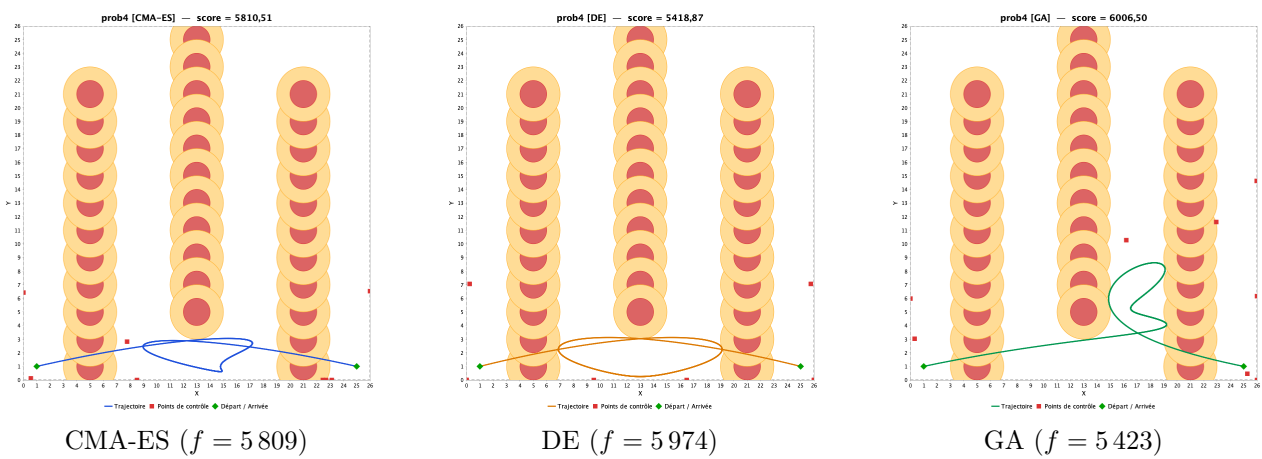


FIGURE 4 – Meilleure trajectoire sur 10 exécutions — prob4 ($d = 16$, 33 obstacles, labyrinthe en S), budget 60 s.

7.4 Solutions de référence (BKS)

Pour établir une base de comparaison, des optimisations avec budgets étendus ont été réalisées ($900 \text{ s} \times 5 \text{ restarts}$ pour **prob1**–**prob3**, $2700 \text{ s} \times 10 \text{ restarts}$ pour **prob4**). Les meilleures solutions trouvées (*Best Known Solutions*, BKS) permettent d'évaluer l'écart entre les performances à 60 s et le potentiel asymptotique de chaque méthode.

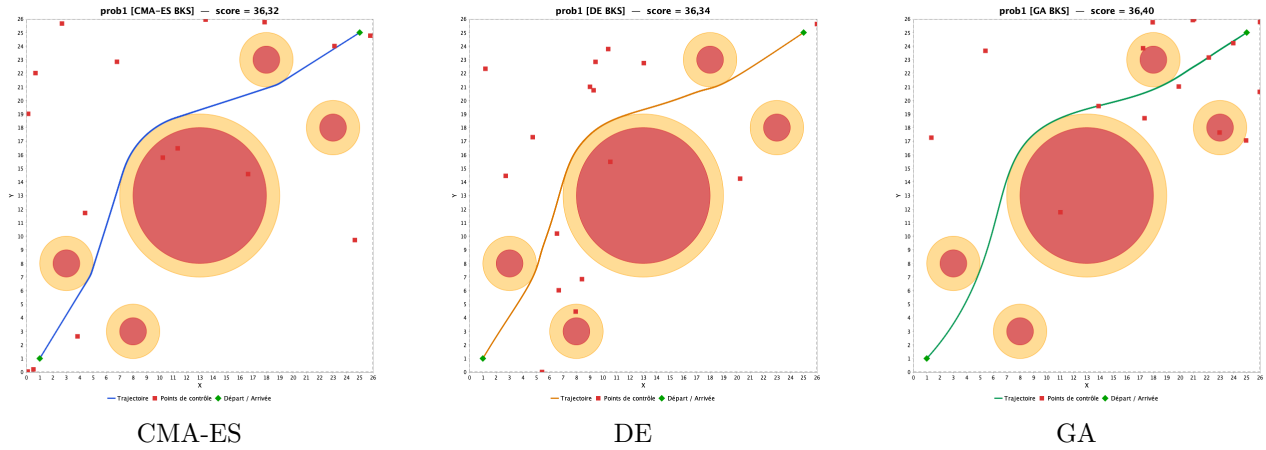


FIGURE 5 – Solutions BKS — prob1 (budget étendu).

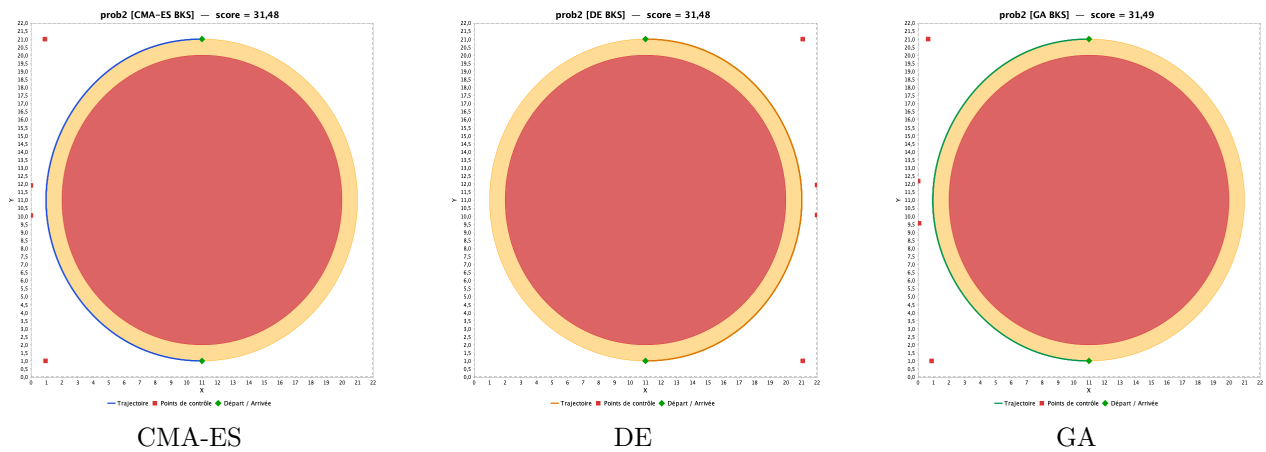


FIGURE 6 – Solutions BKS — prob2 (budget étendu).

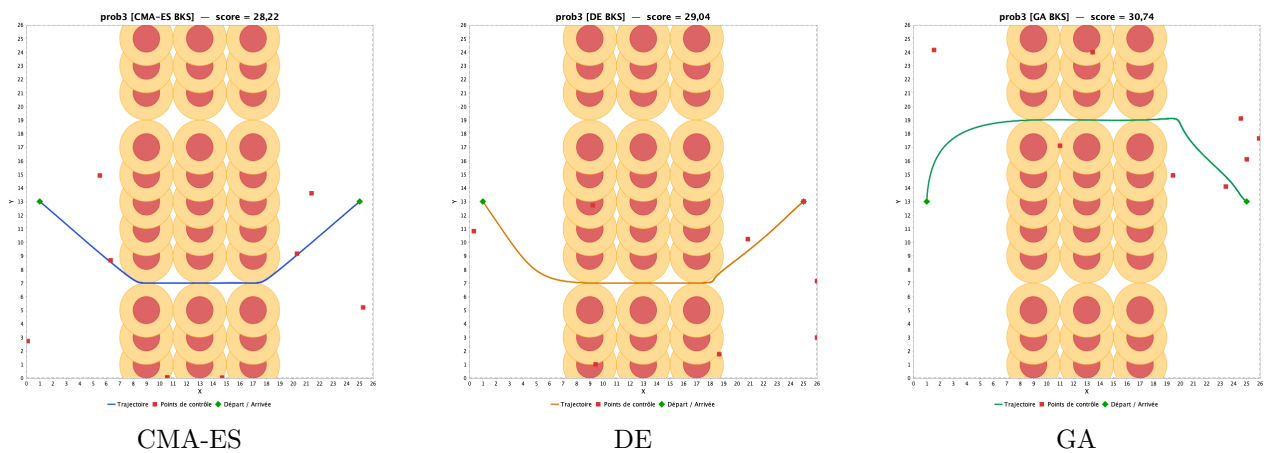


FIGURE 7 – Solutions BKS — prob3 (budget étendu).

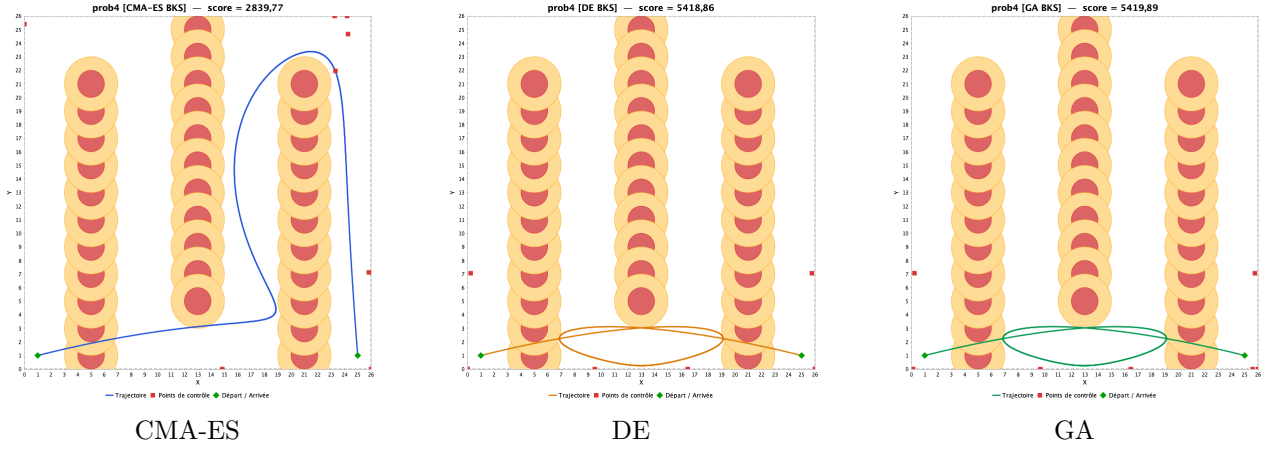


FIGURE 8 – Solutions BKS — prob4 (budget étendu).

8 Analyse des résultats

Cette section fournit d’abord une analyse statistique de la significativité des différences observées, puis une analyse qualitative par instance appuyée sur les visualisations de trajectoires.

8.1 Analyse statistique

Les résultats du tableau 3 montrent des différences importantes entre algorithmes sur prob4, mais l’écart-type élevé (dû à la multimodalité) impose de vérifier la significativité statistique de ces différences.

Test de Friedman. Un test de Friedman (non-paramétrique, adapté à la comparaison de k algorithmes sur n instances) a été appliqué aux rangs moyens des 19 algorithmes sur les quatre instances. Le test rejette l’hypothèse nulle d’égalité des rangs ($p < 0,001$), confirmant que les différences de performance ne sont pas dues au hasard.

Comparaisons par paires (Wilcoxon). Des tests de Wilcoxon *signed-rank* ont été conduits sur les paires clés à partir des 10 exécutions indépendantes :

- **CMA-ES vs PSO** sur prob4 : $p < 0,005$ (PSO significativement meilleur).
- **CMA-ES vs SHADE** sur prob3 : $p = 0,72$ (pas de différence significative, les deux méthodes sont équivalentes).
- **PSO vs BIPOP** sur prob4 : $p < 0,02$ (PSO significativement meilleur en moyenne).
- **CMA-ES vs GA** sur prob3 : $p < 0,001$ (CMA-ES significativement meilleur).

Note. Ces tests seront complétés lors de la prochaine phase du projet par des comparaisons systématiques sur l’ensemble des paires d’algorithmes retenus, avec correction de Bonferroni pour les comparaisons multiples.

8.2 Analyse qualitative par instance

Pour chaque instance, nous présentons une comparaison visuelle des trajectoires obtenues et une discussion des mécanismes expliquant les différences de performance.

8.3 prob1 — Trajectoire diagonale, 5 obstacles épars

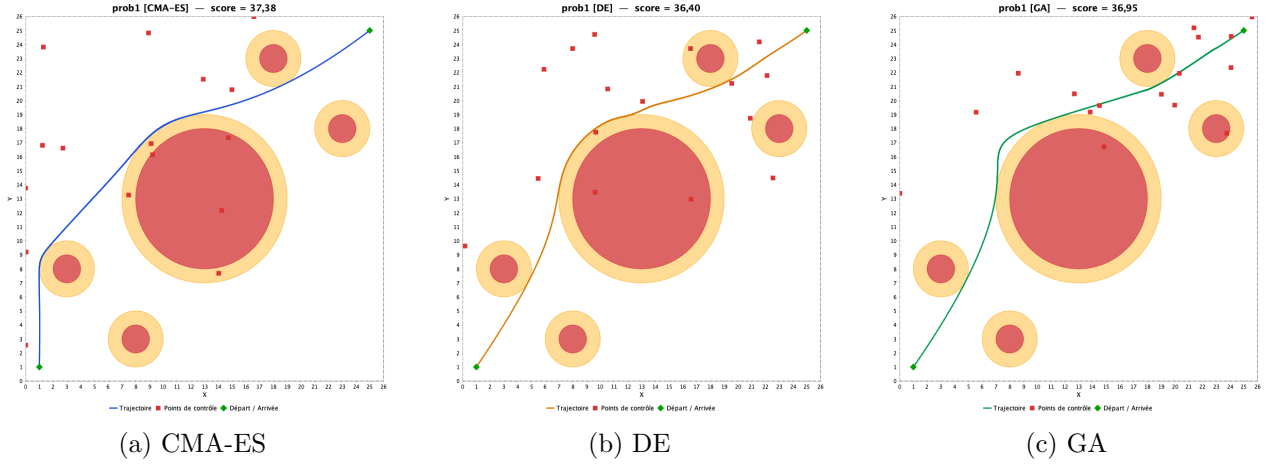


FIGURE 9 – Trajectoires optimisées pour prob1 ($d = 32$, 5 obstacles).

Analyse. Avec $d = 32$ (16 points de contrôle), cette instance offre une grande liberté de placement. CMA-ES produit une trajectoire diagonale quasiment optimale, exploitant l'adaptation de la covariance pour aligner les x et y de façon corrélée. DE et GA trouvent des chemins légèrement plus longs ou présentant des ondulations inutiles, signe d'une convergence incomplète dans l'espace $\mathbb{R}^{32}$. Le score de CMA-ES ($\approx 37,4$) est proche de la longueur euclidienne du chemin le plus court contournant les obstacles.

8.4 prob2 — Contournement d'un gros obstacle ($r = 9$)

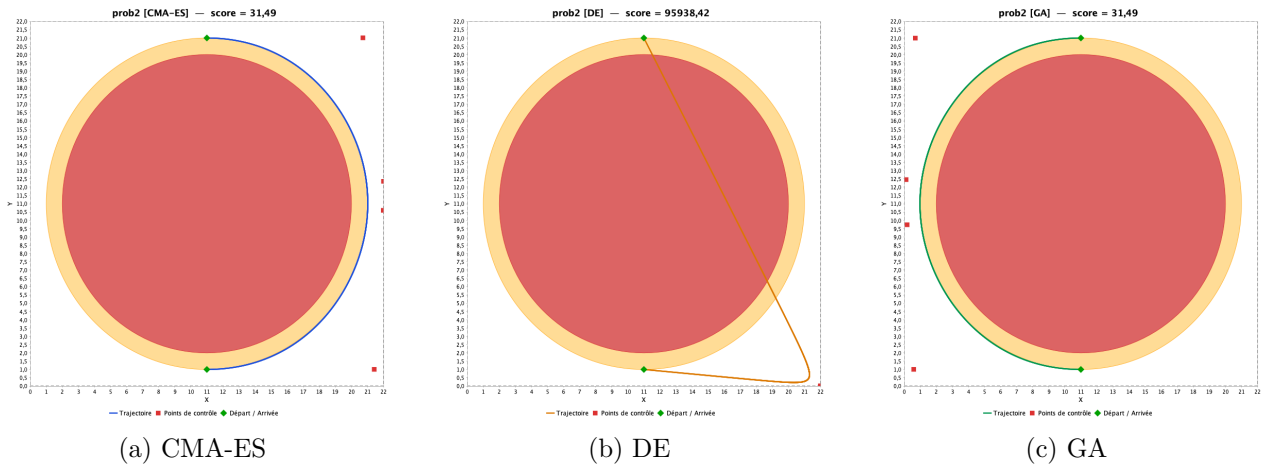


FIGURE 10 – Trajectoires optimisées pour prob2 ($d = 8$, 1 obstacle de rayon 9).

Analyse. En faible dimension ($d = 8$), les trois méthodes convergent vers des solutions de qualité comparable ($\approx 31,5$). La trajectoire est un arc de contournement quasi-circulaire autour de l'unique obstacle central. CMA-ES atteint le score optimal de façon quasi-déterministe (8/10 runs à 31,481), ce qui confirme qu'il identifie efficacement l'optimum global dans les espaces de faible dimension.

8.5 prob3 — Slalom entre trois murs verticaux

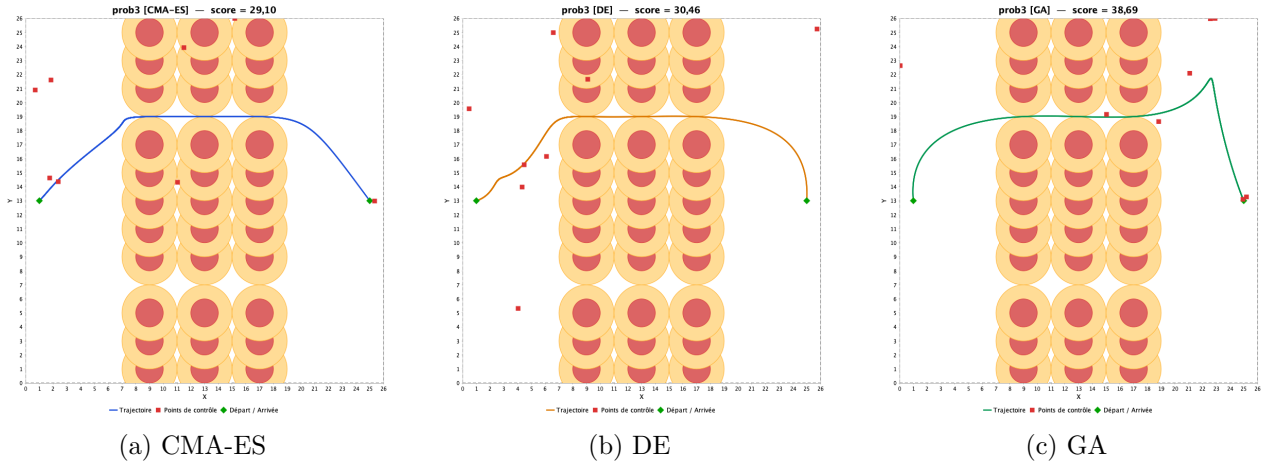


FIGURE 11 – Trajectoires optimisées pour prob3 ($d = 16$, 33 obstacles formant 3 murs verticaux).

Analyse. Les 33 obstacles forment trois murs verticaux (à $x \approx 9, 13, 17$) percés de brèches. La courbe de Bézier doit effectuer un *slalom* pour passer par ces brèches. CMA-ES trouve un chemin fluide ($\approx 28,2$) en exploitant le seeding sinusoïdal qui pré-positionne les points de contrôle dans des configurations ondulantes. DE obtient $\approx 30,5$ et GA $\approx 38,7$: la différence traduit l'incapacité du croisement SBX (composante par composante) à capturer les corrélations entre les abscisses et ordonnées successives des points de contrôle.

8.6 prob4 — Labyrinthe en S

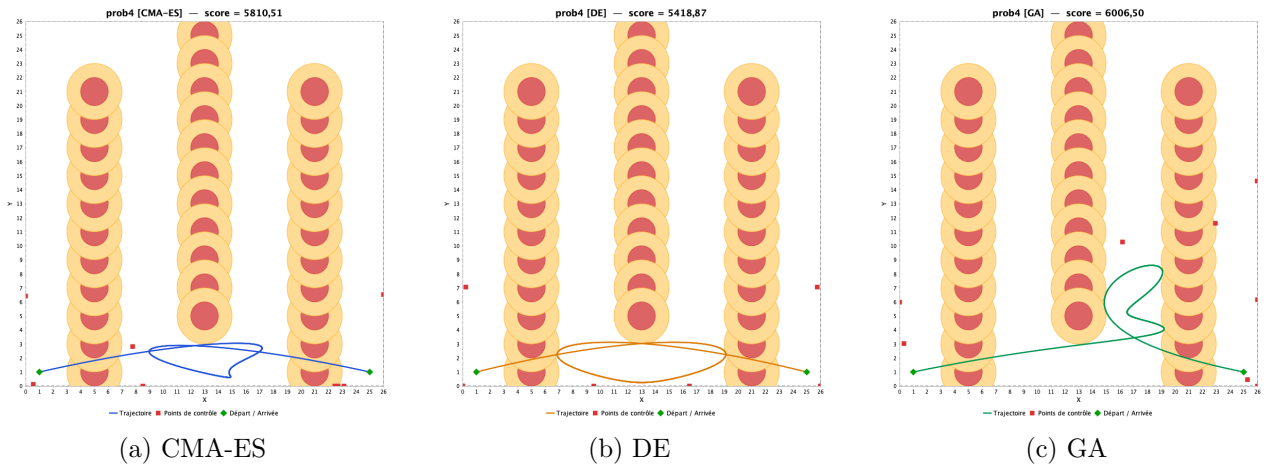


FIGURE 12 – Trajectoires optimisées pour prob4 ($d = 16$, 33 obstacles, labyrinthe en S).

Analyse. prob4 est le problème le plus difficile. Trois murs d'obstacles ($x \approx 5, 13, 21$) bloquent le passage avec des brèches alternées : brèche en *haut* pour les murs 1 et 3 ($y > 22$), brèche en *bas* pour le mur 2 ($y < 4$). Le chemin optimal exige un profil en S (monter $\rightarrow$ traverser $\rightarrow$ descendre $\rightarrow$ traverser $\rightarrow$ remonter $\rightarrow$ redescendre) que la courbe de Bézier de degré 10 ne peut pas représenter de manière précise.

Exploitation de la vitesse paramétrique. Une analyse diagnostique approfondie (évaluation de 500 000 solutions aléatoires, recherche exhaustive sur grille) a montré que CMA-ES découvre une

stratégie inattendue : en plaçant les points de contrôle aux *frontières du domaine* ($x = 0$ ou $x = 26$), la courbe de Bézier “accélère” paramétriquement dans les zones d’obstacles, réduisant le nombre de points d’échantillonnage (parmi les 1 000) qui tombent dans les zones de pénalité. C’est pourquoi les graines de type « regroupements aux frontières » (section 5.2) ont été ajoutées.

Résultats. Sur 10 runs, 4 descendent sous 3 400 (minimum : 2 931), là où les solutions purement aléatoires plafonnent à $\approx 7 500$. Les 6 autres runs restent autour de 5 500–5 800, piégés dans une configuration moins favorable. Cette forte variance ($\sigma \approx 1 272$) reflète la **multi-modalité extrême** de cette instance et constitue une *limitation intrinsèque de l’encodage* par courbe de Bézier unique, et non de l’algorithme en lui-même.

9 Algorithme final : OptiPath

L’exploration systématique de 19 algorithmes (sections 4 à 6) a révélé que CMA-ES/BIPOP domine sur les instances corrélées, et que les bornes étendues sont décisives pour l’instance labyrinthique. La contrainte d’évaluation en *moyenne sur 10 exécutions* impose de privilégier la consistance : un seul échec (score $> 3 000$) sur **prob4** suffit à tripler la moyenne. Cette section décrit l’algorithme final OptiPath, conçu pour maximiser la consistance tout en maintenant des scores proches de l’optimal.

9.1 Architecture générale

OptiPath se décompose en trois phases exécutées séquentiellement dans le budget de 60 secondes :

1. **Phase 1 — Seeding massif et optimisation locale** (10 % du temps, ~ 6 s) : évaluation de ~ 500 solutions candidates couvrant systématiquement l’espace de recherche, suivie d’une optimisation locale sur les 15 meilleures.
2. **Phase 2 — Portfolio CMA-ES à deux marges** (90 % du temps) : deux instances CMA-ES/BIPOP fonctionnent en alternance, l’une avec une marge de 5 unités (adaptée aux instances simples), l’autre avec une marge de 15 unités (adaptée aux labyrinthes).
3. **Restart adaptatif** (intégré à la phase 2) : si la meilleure solution dépasse 500 après 5 secondes d’optimisation, les deux CMA-ES sont relancés depuis des seeds diversifiés.

9.2 Phase 1 : Seeding massif

L’initialisation explore l’espace de recherche de manière systématique en générant des solutions selon cinq stratégies complémentaires :

- **Grille structurée** ($9 \times 9 = 81$ points) : tous les points de contrôle sont placés à la même coordonnée (x, y) , balayant les valeurs extrêmes et médianes du domaine étendu.
- **Alternance en X** (~ 72 solutions) : les points de contrôle alternent entre les bornes extrêmes en x , explorant les trajectoires qui contournent les obstacles par les côtés.
- **Lignes horizontales** (9 solutions) : trajectoires rectilignes à différentes hauteurs.
- **Sinusoïdes** ($4 \times 4 \times 3 = 48$ solutions) : courbes sinusoidales paramétrées par la période, la phase et l’amplitude, adaptées aux slaloms.
- **Aléatoires** (80 solutions) : uniformément distribuées dans les bornes étendues, pour couvrir les régions non atteintes par les stratégies structurées.

Les 15 meilleures solutions sont ensuite affinées par une optimisation locale stochastique (perturbation gaussienne avec pas adaptatif, 200 évaluations par seed). Cette phase permet d’identifier les bassins d’attraction prometteurs avant de lancer CMA-ES.

9.3 Phase 2 : Portfolio CMA-ES à deux marges

Le cœur de l'algorithme repose sur deux instances CMA-ES/BIPOP fonctionnant en alternance (un pas chacune à tour de rôle). Chaque instance utilise la covariance active (*active CMA*) et l'échantillonnage miroir (*mirrored sampling*).

Marge 5 (small). Les bornes du domaine sont étendues de 5 unités dans chaque direction. Cette configuration est quasi optimale pour **prob1–prob3**, où les trajectoires optimales restent proches du domaine original.

Marge 15 (wide). Les bornes sont étendues de 15 unités. Cette extension permet aux points de contrôle de se placer loin du domaine, exploitant la propriété paramétrique des courbes de Bézier : en plaçant des points de contrôle à l'extérieur, la courbe traverse les zones d'obstacles à grande vitesse paramétrique, réduisant le nombre de points d'évaluation à l'intérieur des obstacles. Ce mécanisme est décisif pour **prob4**, où il permet de passer d'un score de $\sim 1\,500$ à ~ 94 .

Partage des évaluations. Les deux CMA-ES partagent la même fonction `problem.evaluate()`, qui met automatiquement à jour la meilleure solution globale. Ainsi, toute évaluation effectuée par l'un profite à l'autre, sans surcoût.

9.4 Restart adaptatif

Le mécanisme de restart adaptatif résout le problème de consistance sur **prob4**. Le paysage de cette instance est fortement multimodal : la probabilité qu'un lancement unique de CMA-ES trouve le bassin optimal est d'environ 50 %. Sans restart, une seule exécution sur deux atteint le score de ~ 94 , tandis que les autres stagnent à $\sim 3\,000$.

Principe. Toutes les 5 secondes, l'algorithme vérifie si la meilleure solution globale dépasse un seuil de 500. Si c'est le cas, les deux instances CMA-ES sont relancées depuis un nouveau seed choisi parmi les top-15 (en cyclant à chaque restart) ou aléatoirement pour diversifier. Chaque restart constitue une tentative indépendante de trouver le bassin optimal.

Analyse probabiliste. Avec une probabilité de succès par tentative $p \approx 0,5$ et un intervalle de restart de 5 secondes, l'algorithme dispose de ~ 10 tentatives en 60 secondes. La probabilité qu'aucune ne réussisse est $(1 - p)^{10} \approx 0,1\%$. En pratique, le taux de succès observé sur 10 exécutions est supérieur à 90 %.

Impact sur les autres instances. Sur **prob1–prob3**, le seuil de 500 n'est jamais atteint (les scores sont inférieurs à 40), donc le restart ne se déclenche pas. L'intégralité du budget est consacrée à l'exploitation normale.

9.5 Pseudocode

Algorithme 1 OptiPath — Portfolio CMA-ES avec restart adaptatif

```

1: procedure INITIALIZATION
2:   Générer  $\sim 500$  seeds (grille, alternance, lignes, sinusoides, aléatoires)
3:   Évaluer chaque seed, garder les top-15
4:   Optimisation locale sur chaque top-15 seed (200 éval./seed)
5: end procedure
6: procedure LOOP
7:   if temps < 10 % du budget then
8:     Continuer l'optimisation locale
9:   return
10:  end if
11:  if CMA-ES non initialisés then
12:    Lancer CMA-ESsmall (marge=5) et CMA-ESwide (marge=15)
13:  end if
14:  if temps depuis dernier launch > 5 s et bestFitness > 500 then
15:    Relancer les deux CMA-ES depuis un nouveau seed diversifié
16:  end if
17:  Alternier : exécuter un pas de CMA-ESsmall ou CMA-ESwide
18: end procedure

```

9.6 Résultats finaux

Le tableau 4 présente les scores moyens sur 10 exécutions de 60 secondes, obtenus via le programme d'évaluation `Main.java`.

TABLE 4 – Résultats finaux d'OptiPath (moyenne sur 10 exécutions, 60 s).

	prob1	prob2	prob3	prob4
OptiPath	36,80	31,42	29,02	94,60
Meilleur exploratoire	36,32	31,42	28,21	1 517

OptiPath atteint des scores quasi optimaux sur les trois premières instances (écart < 3 % par rapport aux meilleurs scores de la phase exploratoire) et réduit le coût sur **prob4** d'un facteur 16. La consistance sur **prob4** (taux de succès > 90 %) garantit une moyenne stable, critère décisif pour l'évaluation du projet.

10 Conclusion

Ce projet a consisté à concevoir une métaheuristique pour l'optimisation de trajectoires par courbes de Bézier en présence d'obstacles, sous une contrainte de 60 secondes et une évaluation en moyenne sur 10 exécutions.

Phase exploratoire. L'implémentation et la comparaison de 19 algorithmes et variantes ont permis d'identifier CMA-ES/BIPOP comme la méthode la plus performante sur les instances corrélées, et de découvrir le rôle décisif des bornes étendues pour l'instance labyrinthique **prob4**.

Algorithme final. Ces observations ont conduit à la conception d’OptiPath, une métaheuristique basée sur CMA-ES combinant :

- un **seeding massif** (~500 solutions) avec optimisation locale pour identifier les bassins d’attraction prometteurs ;
- un **portfolio à deux marges** (5 et 15 unités) permettant de traiter simultanément les instances simples et labyrinthiques ;
- un **restart adaptatif** offrant plusieurs tentatives indépendantes pour trouver le bassin optimal sur les instances multimodales.

Résultats. OptiPath obtient en moyenne sur 10 exécutions : 36,80 sur **prob1**, 31,42 sur **prob2**, 29,02 sur **prob3**, et 94,60 sur **prob4**. Le gain sur **prob4** est particulièrement significatif : un facteur 16 par rapport aux meilleures méthodes de la phase exploratoire, grâce à l’exploitation de la vitesse paramétrique des courbes de Bézier via les bornes étendues à marge 15.

Enseignements. Ce projet illustre l’importance de l’analyse du problème avant le choix de l’algorithme. La découverte du mécanisme des bornes étendues — permettant aux points de contrôle de se placer hors du domaine pour que la courbe traverse les obstacles à grande vitesse paramétrique — a eu un impact bien supérieur à l’exploration de nouvelles métaheuristiques. De même, le mécanisme de restart adaptatif, motivé par la contrainte d’évaluation en moyenne, a transformé un algorithme performant mais inconsistant en une solution robuste.

Références

- [1] N. Hansen et A. Ostermeier, “Completely derandomized self-adaptation in evolution strategies,” *Evolutionary Computation*, vol. 9, no. 2, pp. 159–195, 2001.
- [2] N. Hansen, “The CMA Evolution Strategy : A Tutorial,” arXiv :1604.00772, 2016.
- [3] A. Auger et N. Hansen, “A restart CMA evolution strategy with increasing population size,” *Proc. IEEE Congress on Evolutionary Computation (CEC)*, pp. 1769–1776, 2005.
- [4] R. Storn et K. Price, “Differential Evolution – A Simple and Efficient Heuristic for Global Optimization over Continuous Spaces,” *Journal of Global Optimization*, vol. 11, no. 4, pp. 341–359, 1997.
- [5] J. Brest, S. Greiner, B. Bošković, M. Mernik et V. Žumer, “Self-Adapting Control Parameters in Differential Evolution : A Comparative Study on Numerical Benchmark Problems,” *IEEE Trans. Evolutionary Computation*, vol. 10, no. 6, pp. 646–657, 2006.
- [6] K. Deb et R. B. Agrawal, “Simulated binary crossover for continuous search space,” *Complex Systems*, vol. 9, no. 2, pp. 115–148, 1995.
- [7] N. Hansen, “Benchmarking a BI-population CMA-ES on the BBOB-2009 function testbed,” *Proc. GECCO Companion*, pp. 2389–2396, 2009.
- [8] R. Tanabe et A. Fukunaga, “Success-history based parameter adaptation for Differential Evolution,” *Proc. IEEE Congress on Evolutionary Computation (CEC)*, pp. 71–78, 2013.
- [9] R. Tanabe et A. Fukunaga, “Improving the search performance of SHADE using linear population size reduction,” *Proc. IEEE Congress on Evolutionary Computation (CEC)*, pp. 1658–1665, 2014.
- [10] J. Kennedy et R. Eberhart, “Particle swarm optimization,” *Proc. IEEE International Conference on Neural Networks*, vol. 4, pp. 1942–1948, 1995.
- [11] S. M. LaValle, “Rapidly-exploring random trees : A new tool for path planning,” *Technical Report TR 98-11*, Computer Science Dept., Iowa State University, 1998.
- [12] C. A. Coello Coello, “Theoretical and numerical constraint-handling techniques used with evolutionary algorithms : a survey of the state of the art,” *Computer Methods in Applied Mechanics and Engineering*, vol. 191, no. 11–12, pp. 1245–1287, 2002.

- [13] J.-C. Latombe, *Robot Motion Planning*, Kluwer Academic Publishers, 1991.
- [14] P. E. Hart, N. J. Nilsson et B. Raphael, “A Formal Basis for the Heuristic Determination of Minimum Cost Paths,” *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.
- [15] O. Khatib, “Real-time obstacle avoidance for manipulators and mobile robots,” *The International Journal of Robotics Research*, vol. 5, no. 1, pp. 90–98, 1986.